

Tail-W4 效率部分：回应评审 W2-W6

Qwen3.5-4B, 分支 rollout-precision-scheduler-clean-last-try

2026-09-25

摘要：可以直接用的结论和需要你拍板的事

范围。只处理效率部分(评审 W2-W6), Qwen3.5-4B。没有改论文。新代码在分支 rollout-precision-scheduler-clean-last-try, worktree /data/huanchen/ver1-clean-last-try, 三个提交 (第 1 节与第 8 节)。所有数字都是 steps 9-48, 逐 step 配对, 区间同时给逐步独立重抽和长度 4 的滑动块重抽。

已经确定、可以直接用的 (不依赖还在跑的 run)

- W2 计时口径 (第 2、10 节)。**239s (Table 1) 与 259s (Fig. 6) 之差 = reference (KL) 那一遍 17s + 权重同步 4.5s, 实测各分量加总与 step 时间吻合到 0.03s。今晚在不调用 nvidia-smi 的条件下重测, 实测 downstream 与调度器的拟合斜率相差不到 1%, **没有测量污染的迹象**。建议统一用实测完整 step: 4B Tail-W4 $1.26 \rightarrow 1.24\times$; uniform W4 $1.03 \rightarrow 1.00\times$ (正文 “little step-level benefit” 要改成 “no”); 附录 A.8 “directly measured downstream” 的说法需要改正。
- W3 准备成本 (第 3 节)。**4B 同配置实测 87min (BF16 校准 34, uniform-W4 24, tail-W4 续跑 22, TPOT 8), 按实际做法约 **84 步回本**; 改用前 8 个训练步的 BF16 rollout 并去掉 uniform-W4 轨迹后约 38min、36 步回本。附录 “约 3 个 step” 应替换。相同准备预算的比较对 Tail-W4 有利: 用 pilot 挑固定 K 要丢掉约 266min 的训练, 是 Tail-W4 准备的 3 倍。
- W5 预测有效性 (第 4 节)。**只用校准时, 模型把最优切换点排在 $K = 3000$ (与 8 个固定 K 实测排序的 Spearman +0.33), 实际上 3000 比最优差 5%; 在线学习后排序变准 (+0.57 到 +0.67), 预测最优点进入实际的平坦带。BF16 排空曲线平均误差 $\leq 0.6/32$ 条, 但逐 rollout 抓不住波动。
- W6 显存 (第 5 节)。**双精度常驻使 KV 容量减少 11% ($853,580 \rightarrow 759,883$ token), 但 B32 下峰值只用到了其 17%, 没有任何抢占。
- 独立重复 (第 8 节)。**旧代码 Tail-W4 四个 seed 实测 $1.202 \pm 0.028\times$, 论文写的 seed 42 (1.24) 是最幸运的一个; 同口径最好的固定 K 为 $1.189\times$ 。

今晚发现、会影响论文表述的

- FSDP2 (KL 关) 下自适应与固定策略打平 (第 7 节)。**Tail-W4、 $K=2750/5000/7000$ 、No online update 全在 $1.20-1.24\times$, 两两差距都不显著; 只用校准挑的 $K=2750$ 就和 Tail-W4 一样好。论文的 DP 与长训练两节是 FSDP2: 相对 BF16 的加速不受影响, 但不暗示自适应相对简单规则有额外收益。
- 归组 bug 与它掩盖的第二个问题 (第 1、8 节)。**在线更新把 cohort 送进了错误的 W4 组; 只修这一处, Megatron 下反而变慢 ($259 \rightarrow 272s$), 因为模型随即逃到第一个 tail cut 之下、由未学习的 uniform-W4 组定价的 2250。旧代码里两个错误恰好互相抵消。第二个修复禁止在第一个 tail cut 之下切换。

新代码的结果 (第 9 节): 没有变好。加了下限的新代码在 seed 42 上比旧代码显著慢 8% (旧/新 $0.919 [0.851, 0.988]$), seed 43、44 与旧代码无显著差别; 它在 seed 42 上有一半 rollout 贴着下限 2750 切换, 和 No online update 一样。归组修好之后, 在线学习把决策推向早切; 旧代码里这个推动被 bug 挡住了。**建议论文保留现有代码的结果。**剩余 GPU 时间已改为给现有代码的三个 ablation 补 seed 43/44 (第 8.2 节已有的单 seed 结果中, No online update 与 Constant TPOT 已经显著)。

需要你决定

- Table 1 用实测完整 step (含 reference, 建议) 还是拟合口径 (需在 A.8 如实写明不含 reference 与同步)。
- 4B 主结果: 建议保留现有代码的数字 (seed 42), 同时在正文或附录报告四个 seed 的平均 ($1.202 \pm 0.028\times$ 实测)。“比固定策略好” 建议改成 “与事后最优的固定 K 持平, 并显著优于相同准备预算下可得固定规则”: 只用校准挑的固定点在已测 K 中最接近的是 $K=3000$, 与现有代码 seed 42 配对慢 $1.100\times [1.050, 1.156]$; pilot 挑的 $K^* = 11000$ 慢 $1.063\times [1.000, 1.134]$ (块重抽 $[1.010, 1.121]$)。

3. 是否在局限性中说明在线更新的归组问题及修复后的表现（第 9 节）。
4. 是否在正文承认 FSDP2 下自适应无额外收益。

Contents

1 代码改动: rollout-precision-scheduler-clean-last-try	2
2 W2: 计时口径	2
3 W3: 包含准备成本的净收益	3
4 W5: 预测模型是否有效	4
5 W6 (部分): 显存、KV 容量与复现参数	5
6 今晚的实验设计	5
7 FSDP2 (KL 关): 自适应与固定策略打平	5
8 Megatron (KL 开, 论文配置): 修复、它暴露的问题、第二个修复	7
8.1 旧代码的真实水平: 四个 seed 平均与最好的固定 K 持平	7
8.2 论文现有代码: ablation 与固定策略的配对差值区间 (回应 W6)	7
8.3 只修归组 (a703ca55): 变慢了, 原因是逃到第一个 cut 之下	7
8.4 第二个修复 (cc1eea4e): 不在第一个 tail cut 之下切换	8
9 新代码 (cc1eea4e, Megatron) 的结果	9
10 W2 检验: 实测 downstream 是否被污染	10
A 今晚的 run 清单	10

1 代码改动: rollout-precision-scheduler-clean-last-try

分支从 rollout-precision-scheduler-clean 的 f8cf7f21 (论文所有 Megatron run 使用的调度器代码) 出发, 加了三个提交 (第三个 cc1eea4e 见第 8 节)。每个新 run 目录下的 `verl_commit.txt` 记录了它用的提交; 后缀 fsdp/mg 为 a703ca55, mgf 为 cc1eea4e, old 为论文代码 f8cf7f21。vLLM 侧没有改动: vllm-clean 最后一次代码提交是 9 月 17 日, 论文的 BF16、固定 K 、clock、=0 run 都在那之后。

986916e8: 按切换位置给 W4 cohort 归组 (bug 修复)。 在线更新把每批切到 W4 的请求 (cohort) 交给一个 W4 组学习。旧代码按这批请求进入位置的中位数选组, 而进入位置比实际切换位置早几个 token (在 2750 切, 记录的是 2746-2750)。中位数一旦是 2748, 就小于 2750 这个 cut, 于是这批数据被送进了 cut 0 组, 而给这次切换定价的是 2750 组: **定价的组学不到自己决策产生的数据**。seed 42 前 14 个 rollout 里只有 2 批进了 2750 组。现在按 `trigger.applied_response_tokens` (每条记录都有) 归组。用 seed 42 的真实状态重放: 修复后计划在第 4 次修订就离开 2750, 旧代码要到第 14 次; 最终落点相同 (6750 $\rightarrow$ 7250)。旧归组的重放与线上实际决策逐行一致, 重放重建的决策表与线上 `policy.json` 有 98.5% 的格子完全相同 (其余由每次修订最多移动 2000 token 的限速器解释)。

a703ca55: 相邻 cut 之间共享强度证据 (选项, 默认关闭)。 `--w4-share-tokens S`: 一批数据除了更新给它定价的组, 还按 $1 - |F - \text{cut}|/S$ 的权重更新附近的组, 只共享强度参数, 各组保留自己的校准形状。离线评估 (8 条已有 clock run, 每个 rollout 用它之前的表预测): 对 4B、9B 有帮助, 对两个 Phi 过冲, 所以默认关闭, 今晚不用。

2 W2: 计时口径

评审指出 Table 1 的 4B Tail-W4 是 239s, 而 Fig. 6 约 260s。拆开之后 (图 1) 答案很明确:

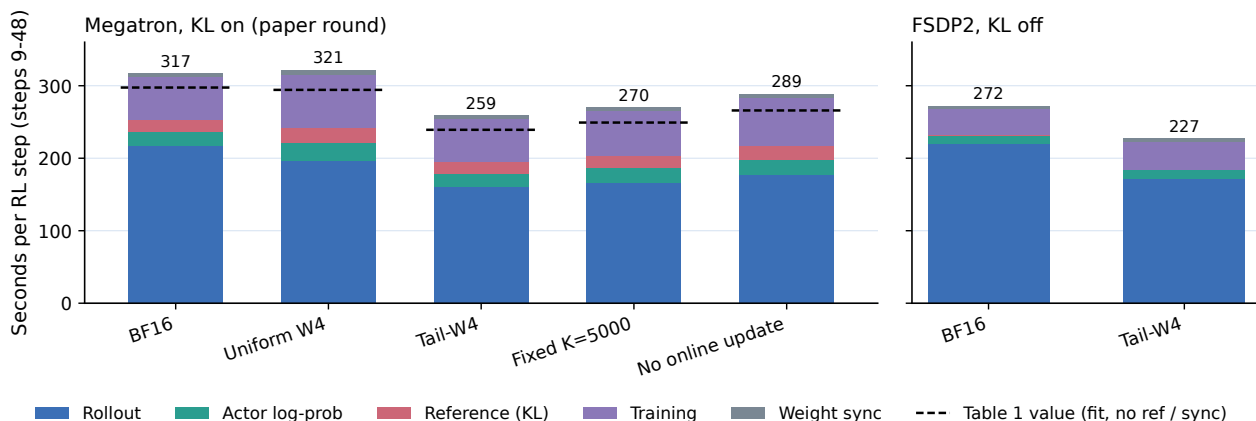


Figure 1: 4B 每个 arm 的分阶段时间 (steps 9-48)。虚线是 Table 1 的数：实测 rollout 加上不含 reference 的 downstream 拟合。

- 实测 step 时间等于各阶段之和，“其他”一项平均只有 0.02-0.03s，没有未计入的时间。
- Table 1 的 downstream 用的是不含 *reference* 的拟合 (0.494ms/token + 4.6s，与 `dp_resample.py` 的 `FITS["paper"]` 一致，九个格子全部复现到打印位)。它和实测的 log-prob + 训练只差约 2.5s。
- 实测比拟合多出的约 20s，正好是 **reference (KL) 那一遍 (约 17s) 加权重同步 (约 4.5s)**。这些 run 开着 KL，这两项是真实发生的工作。

因此 239s 对应“不含 reference 和同步”，259s 对应完整 RL step。附录 A.8 写的“Table 1 报告的是直接测量的 downstream”与实际做法不符，这是评审 W2 抓住的地方。

两种口径下的 4B 结论：Tail-W4 相对 BF16 (seed 42+43 合并) 在拟合口径下是 1.261 $\times$ ，在实测完整 step 口径下是 1.240 $\times$ ；Tail-W4 相对固定 K 和各 ablation 的比值在两种口径下几乎不变 ($K=5000$: 1.042 vs 1.043; No online update: 1.112 vs 1.114; 差值都在 0.003 以内)。变化最大的是 uniform W4: 相对 BF16 从拟合口径的 1.025 $\times$ 变成实测口径的 0.999 $\times$ ，因为它多生成 20% 的 token，reference 那一遍也跟着变贵。换用实测口径时，正文“uniform W4 offers little step-level benefit”要改成“offers no step-level benefit”。

关于“**实测 downstream 被 nvidia-smi 问题污染**”。我在代码、文档和日志里都没有找到这个问题的记录；实测各分量彼此自洽，而且差值能被 reference 和同步完全解释。今晚的新 run 在运行期间不调用 `nvidia-smi`，并且关掉了 KL (见第 8 节)，可以直接检验：若实测 downstream 与 FSDP2 拟合一致，就说明之前的差距来自 reference 而不是污染。结果见第 10 节。

建议：最干净的做法是整篇统一用实测完整 step。今晚的新一轮用 FSDP2 并关掉 KL，此时没有 reference 那一遍，实测完整 step 就是要报告的数，不需要任何拟合，也与论文 DP 和长训练两节 (本来就是 FSDP2) 口径一致。

3 W3: 包含准备成本的净收益

附录 A.11 的 1,084s (约 3 个 BF16 step) 是跨配置的组件估计，而且假设复用 BF16 训练 rollout。4B 在同一台机器、同一配置下的实测 (图 2a): BF16 校准 33.8min, uniform-W4 轨迹 24.2min, tail-W4 续跑 21.5min (293 条, 1.74M token), TPOT 测量 7.8min, 合计 **87 min** (只算生成时间, 不含每个任务的启动)。Tail-W4 每步比 BF16 省 62s (实测完整 step, Megatron), 所以**按实际做法要约 84 步回本**, 比附录的估计悲观得多。如果把 BF16 校准换成直接使用前 8 个训练步的 BF16 rollout、并省掉 uniform-W4 轨迹 (它只为 cut 0 以下的切换定价), 准备成本降到约 38min, 回本约 36 步。

评审真正要求的是**相同准备预算**下的比较。用 pilot 挑固定 K (论文 Fig. 10 的做法: 8 个候选各跑 8 步, 留下最好的一个) 要丢掉 7 条候选的 56 个训练步, 约 266min, 是 Tail-W4 准备成本的 3 倍。只用同一份校准数据挑一个固定 K 则几乎零额外成本, 但挑出来的是 2750 (第 4 节解释了为什么校准会挑早), 今晚实测它的表现 (第 8 节)。

建议的表述：把 A.11 改成同配置实测，给出两个回本点 (按实际做法、按复用 BF16 步)，并把“准备成本 vs pilot 选择预算”作为主要比较；不要再写“约三个 step”。

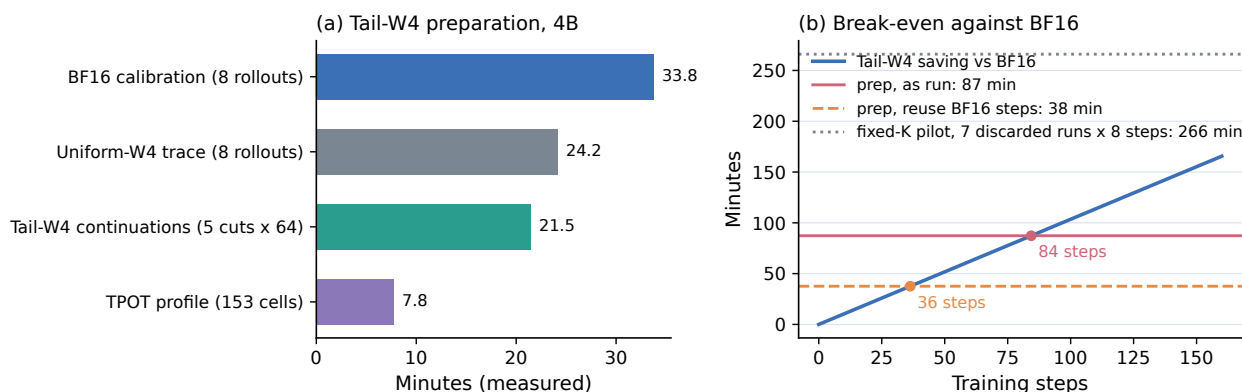


Figure 2: (a) 4B 同配置实测的准备时间。(b) 相对 BF16 的回本点，以及用 pilot 挑固定 K 所浪费的训练预算。

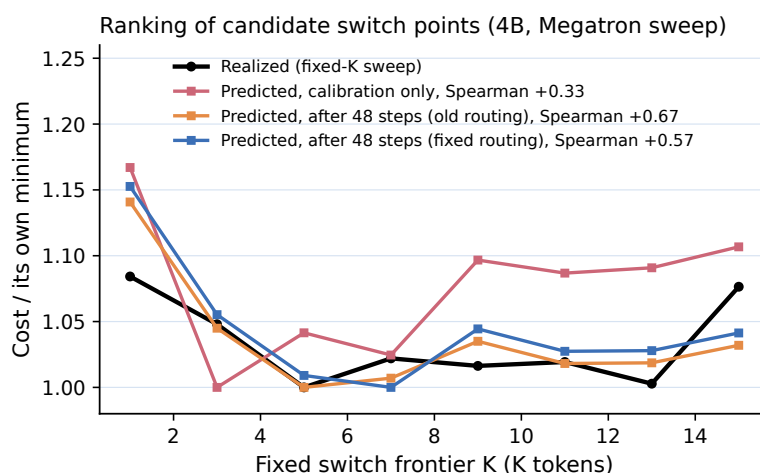


Figure 3: 候选切换点的排序质量。黑线：8 个固定 K arm 在同一批 prompt 上的实际成本（Megatron，seed 42）。彩线：代价模型对同样 8 个计划的预测。各自除以自己的最小值。

4 W5：预测模型是否有效

评审要求在留出数据上检验存活数、剩余 token 和候选切换点排序。所有检验都是 held-out：第 r 个 rollout 只用第 $r-1$ 次修订的表来预测。

排序质量（图 3）。 只用校准时，模型把最优点排在 $K = 3000$ （Spearman +0.33），而实际上 3000 比最优差 5%，真实的平坦带在 5000–13000。学了 48 步之后，预测的形状贴住了实际曲线（Spearman +0.67 旧归组、+0.57 修复后），预测最优点移进了平坦带。这正是 No online update 永远停在 2750、在 Megatron 下损失最大的原因。逐步的排序相关只有约 +0.1：单个 rollout 的成本噪声很大，模型能抓住的是平均意义上的形状。预测的绝对成本比实际高约 15%，但决策只依赖排序。

BF16 排空曲线。 切换之前整段是 BF16，所以存活请求数完全可观测。到 6K token 为止，预测的平均存活数与实际相差不超过 0.6 条（共 32 条）；但逐个 rollout 的实际值在一个很宽的带里（2.75K 处从 7 到 25 条），预测几乎是一条线。单参数的强度模型只抓水平，抓不住 batch 之间的波动；后者取决于这 8 个 prompt 的难度，模型没有任何输入能看到。

W4 续跑长度。 在决策定价的那个量（切换后还要生成多少）上，在线学习把水平往真实值拉近：修复后 8 条 clock run 中 7 条的水平偏差缩小（4B 从约 -15% 到约 -9%），例外是 Phi-4 14B，它的续跑分布形状与校准差太多，单参数模型拉不回来。

校准状态与运行时状态不同 (A.1/A.4)。 这一项需要在 MARLIN 生产路径上做成对的自由生成实验，今晚没有做。

5 W6 (部分): 显存、KV 容量与复现参数

显存与 KV 容量。 同一张 A100-80GB, vLLM 预算 `gpu_memory_utilization=0.5` (40 GiB), 4B:

	模型占用	可用 KV	KV 容量	16K 请求并发
纯 BF16	8.18 GiB	29.36 GiB	853,580 token	52.1×
双精度常驻	11.3 GiB	26.15 GiB	759,883 token	46.3×

INT4 影子权重本身 1.71 GiB; 双精度让 KV 容量减少 11%。在论文的 B32 设置下，每个 rollout 的峰值 KV 需求中位数 68,930 token、最大 129,043 token，只占双精度容量的 17%；BF16、uniform W4、Tail-W4、固定 K 以及 FSDP2 各 run 的日志里都没有一次 KV 抢占。所以在这里容量减少不影响结果；在同样 40 GiB 预算下 batch 大到约 46 个 16K 长请求同时存活时才会开始起作用。训练侧峰值显存 (Megatron actor) 三种精度都在 20.4–21.0 GiB。(来源: TPOT heatmap 测量时两个引擎的 vLLM 日志; 训练侧来自 actor/perf/max_memory_allocated_gb。)

复现参数。 clock 更新: 每条线一个强度参数 $a = (\kappa + D)/(\kappa + E)$, $S = S_0^a$; BF16 线 $\kappa = 64$, W4 组 $\kappa_g = 64 \times n_g/256$ (4B: cut 2750/5000/6250 各 $n_g = 64$, 即 $\kappa_g = 16$; 7250 为 14; 8250 为 11.2; cut 0 为 64); 每个 rollout 折扣 $\rho = 0.9$; 决策每次修订最多移动 2000 token; 校准为 256 条 BF16 请求 + 5 个 cut (存活比例 1/2–1/6) 各 64 条 W4 续跑 + 256 条 uniform-W4; TPOT 表 7 个 batch $\times$ 11 个上下文长度; 网格步长 250 token; downstream 斜率 Megatron 0.604 ms/token (含 reference), FSDP2 0.328 ms/token (不含)。

6 今晚的实验设计

所有新 run 都是 Qwen3.5-4B、单卡、B32 (8 个 prompt $\times$ 4)、24K cap、48 步、GSM8K, 代码固定在 a703ca55。各 arm 使用同一个 prompt 顺序，所以第 s 步在所有 run 里看到的是同一批 8 个 prompt; 比较两个 arm 时逐 step 配对，在两个 arm 都有的 seed 上合并。区间给两种: 逐步独立重抽，以及长度为 4 的滑动块重抽 (评审 W6 指出相邻训练步不独立，块重抽更保守)。凡是不做在线 W4 学习的 arm (BF16、固定 K 、阈值 t 、uniform W4、No online update)，归组修复不影响它们的行为，直接复用已有 run。

今晚跑了两轮:

- **FSDP2、KL 关** (先跑): 比 Megatron 快约 14%，而且没有 reference 那一遍，W2 的口径问题在这个设置下根本不存在; 论文的 DP 和长训练两节本来就是 FSDP2。
- **Megatron、KL 开** (论文那一轮的配置，第二轮起切换过去): 原因见下一节。

7 FSDP2 (KL 关): 自适应与固定策略打平

Table 1: FSDP2 单卡、KL 关, steps 9–48 的逐 arm 汇总 (实测完整 step)。加速比相对 BF16, 按 seed 配对。

Arm	seeds	step (s)	rollout (s)	响应 token/step	切换点中位数	相对 BF16 [95% CI]
BF16	42	272.6	219.8	153,493	–	–
Tail-W4 (fixed routing)	42	223.4	166.7	160,183	2750	1.220 [1.132, 1.318]
Tail-W4 (旧归组)	42,43	227.7	172.4	157,317	4438	–
Fixed $K = 2750$	42	220.1	164.2	161,083	2750	1.239 [1.146, 1.343]
Fixed $K = 5000$	42	223.4	168.7	159,527	5000	1.220 [1.139, 1.313]
Fixed $K = 7000$	42	218.7	165.1	155,645	7000	1.247 [1.153, 1.353]
No online update	42	221.1	164.7	162,471	2750	1.233 [1.138, 1.338]
Constant TPOT (旧归组)	42	224.2	167.6	162,252	2750	–
No state replanning (旧归组)	42	218.4	161.3	161,918	2750	–

Table 2: Tail-W4 相对固定策略（比值 = 对方 / Tail-W4, > 1 表示 Tail-W4 更快）。按 step 配对（同一批 prompt），多个 seed 合并；区间分别来自逐步独立重抽和长度 4 的滑动块重抽。

对比	比值	95% CI (逐步)	95% CI (块)	seeds	显著
Fixed $K = 2750$	0.985	[0.935, 1.036]	[0.950, 1.034]	42	否
Fixed $K = 5000$	1.000	[0.941, 1.068]	[0.945, 1.057]	42	否
Fixed $K = 7000$	0.979	[0.926, 1.038]	[0.927, 1.034]	42	否

Table 3: 各 ablation 相对 Tail-W4（比值 = ablation / Tail-W4, > 1 表示该组件有用）。按 step 配对（同一批 prompt），多个 seed 合并；区间分别来自逐步独立重抽和长度 4 的滑动块重抽。

对比	比值	95% CI (逐步)	95% CI (块)	seeds	显著
No online update	0.990	[0.929, 1.054]	[0.944, 1.019]	42	否

Table 4: 归组修复本身（比值 = 旧归组 / 修复后，同 seed）。按 step 配对（同一批 prompt），多个 seed 合并；区间分别来自逐步独立重抽和长度 4 的滑动块重抽。

对比	比值	95% CI (逐步)	95% CI (块)	seeds	显著
Tail-W4	1.018	[0.958, 1.083]	[0.959, 1.069]	42	否

结论：FSDP2 下成本地形是平的。从 $K=2750$ 到 $K=7000$ ，以及 Tail-W4、No online update，全部落在 BF16 的 $1.20\text{--}1.24\times$ 之间，两两之比都在 $0.95\text{--}1.00$ ，区间全部跨过 1。只用校准数据挑出的 $K=2750$ （准备预算与 Tail-W4 相同）已经和 Tail-W4 一样好。原因是关掉 KL 以后 downstream 便宜了（没有 reference 那一遍），W4 多生成的 token 罚得轻，早切晚切差别不大。**这一点论文有直接影响：**DP 和长训练两节用的是 FSDP2，在那个设置下，“自适应优于固定规则”没有数据支撑。那两节报告的是相对 BF16 的加速，这本身不受影响；但不应暗示自适应相对简单规则有额外收益。

归组修复在 FSDP2 上的效果：旧归组 / 修复后 = 1.018 [0.958, 1.083]，方向正确但不显著。

修复后暴露的另一个问题：低于第一个 tail cut 的切换定价。修复后的完整版在前 13 个 rollout 停在 5250–7000，第 16 个 rollout 起落到 2250 并停住（图 4）。重放各组学到的参数可以看清原因：第 14 次修订时，2750 组学到“切过去以后比较准说的更长”（ a 从 1 降到 0.80），在 2750 切的预测成本从 254s 涨到 286s；而低于 2750 的切换由 cut 0 组（uniform W4，从 prompt 开头就用 W4 的数据）定价，它此时一条数据都没学过，于是 2250 显得最便宜。之后 2250 的数据都喂给 cut 0 组，它学到“更短”（ a 升到 1.2），估价继续下降，锁在 2250。这是“学得最少的组看起来最好”：cut 0 组的先验强度是 64，tail 组只有 16，而 cut 0 组的数据本来就不是“BF16 前缀之后再切到 W4”的续跑。在 FSDP2 上它没有造成损失（地形是平的），但它是一个真实的缺陷。两个修法，都没有在今晚使用：(1) 候选切换点不低于第一个 tail cut（那里校准没有数据支撑）；(2) 让各组先验强度相同。

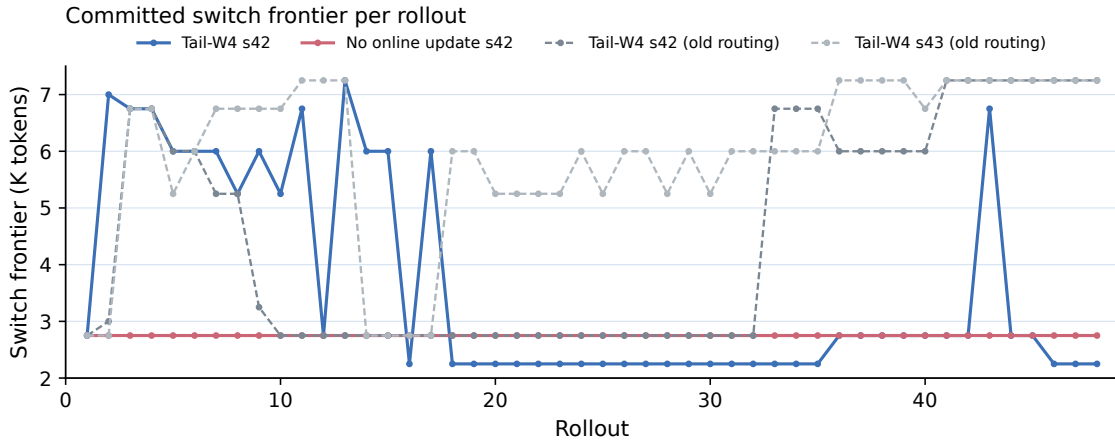


Figure 4: FSDP2 下每个 rollout 的切换点。修复后（实线蓝）在第 16 个 rollout 之后落到 2250。

8 Megatron (KL 开, 论文配置): 修复、它暴露的问题、第二个修复

8.1 旧代码的真实水平: 四个 seed 平均与最好的固定 K 持平

论文 Table 1 的 4B Tail-W4 (1.26 $\times$) 来自 seed 42。旧代码其实已有四个独立 seed (42–45), 按实测完整 step、对 BF16 seed 42+43 合并: 1.240、1.179、1.181、1.206, 平均 1.202 ± 0.028 (拟合口径 1.224 ± 0.027)。seed 42 是四个里最幸运的一个。同口径下最好的固定 K ($K=5000$, seed 42) 是 1.189 $\times$ 。所以按独立重复来看, 旧代码的 Tail-W4 与最好的固定 K 基本持平, 论文 Fig. 7 “on par, overlapping CIs” 的措辞是对的; 要写成“更好”, 需要新代码把平均值抬上去。

8.2 论文现有代码: ablation 与固定策略的配对差值区间 (回应 W6)

评审要求“直接报告 Tail-W4 相对固定规则和各消融的差值区间”。下表全部相对论文现有代码的 Tail-W4 (f8cf7f21), 实测完整 step, 逐 step 配对; No online update 与固定 K 不受代码影响, 所以第二个 seed 可以用今晚的新 run (eff2_*_s43_mg)。

Table 5: 比值 = 对方 / Tail-W4 (> 1 表示 Tail-W4 更快)。Megatron、KL 开, steps 9–48。

对比	比值	95% CI (逐步)	95% CI (块, 长度 4)	seeds
No online update	1.055	[1.017, 1.095]	[1.025, 1.086]	42, 43
No downstream cost	1.056	[0.993, 1.123]	[1.005, 1.100]	42
Constant TPOT	1.049	[1.004, 1.098]	[1.006, 1.088]	42
No state replanning	1.051	[0.992, 1.112]	[0.992, 1.082]	42
固定 $K = 5000$ (事后最优)	1.028	[0.984, 1.078]	[0.992, 1.064]	42, 43
固定 $K = 3000$ (只用校准挑的最近点)	1.100	[1.050, 1.156]	[1.047, 1.153]	42
固定 $K^* = 11000$ (pilot 挑的)	1.063	[1.000, 1.134]	[1.010, 1.121]	42

读法: 不换代码, No online update 与 Constant TPOT 在两种区间下都显著; No downstream cost 只在块重抽下显著; No state replanning 不显著。相对事后最优的固定 K 两个 seed 打平; 相对相同准备预算下可得固定规则 (只用校准挑, 或 8 步 pilot 挑) 显著更好。这些是单一实现、单卡 4B 的结论; 只有 No online update 与 $K=5000$ 有两个 seed。

8.3 只修归组 (a703ca55): 变慢了, 原因是逃到第一个 cut 之下

Table 6: FSDP2 单卡、KL 关, steps 9–48 的逐 arm 汇总 (实测完整 step)。加速比相对 BF16, 按 seed 配对。

Arm	seeds	step (s)	rollout (s)	响应 token/step	切换点中位数	相对 BF16 [95% CI]
BF16	42,43	321.4	220.1	155,971	–	–
Tail-W4 (fixed routing)	42,43	272.3	167.3	159,140	2250	1.194 [1.133, 1.260]
Tail-W4 (旧归组)	42,43,44,45	267.6	164.7	156,356	6000	–
Fixed $K = 3000$	42	285.1	172.9	168,240	3000	1.111 [1.018, 1.213]
Fixed $K = 5000$	42,43	275.0	171.5	157,623	5000	1.179 [1.114, 1.248]
Fixed $K = 7000$	42	277.0	172.0	158,602	7000	1.143 [1.067, 1.228]
Fixed $K = 9000$	42	275.3	173.4	153,652	9000	1.150 [1.067, 1.237]
No online update	42,43	292.1	179.8	172,364	2750	1.111 [1.051, 1.173]
Constant TPOT (旧归组)	42	272.1	168.7	160,463	2750	–
No state replanning (旧归组)	42	272.6	169.7	157,900	6750	–
Fixed $K = 1000$	42	294.1	177.3	176,682	1000	1.077 [1.000, 1.158]
Fixed $K = 11000$	42	275.7	175.1	152,214	11000	1.149 [1.060, 1.243]
Fixed $K = 13000$	42	271.0	172.2	149,877	13000	1.169 [1.090, 1.253]
Fixed $K = 15000$	42	292.1	184.6	161,338	15000	1.084 [1.011, 1.161]
t4	42	289.1	183.3	157,564	250	1.095 [1.012, 1.186]
t8	42	278.0	175.0	156,392	250	1.139 [1.057, 1.216]
t12	42	274.1	169.7	158,089	250	1.155 [1.062, 1.254]
t16	42	283.7	174.6	162,819	250	1.116 [1.034, 1.198]
uniform	42	321.7	197.0	187,512	–	0.985 [0.917, 1.051]
No downstream cost (旧归组)	42	273.7	170.0	156,940	5250	–

Table 7: Tail-W4 相对固定策略（比值 = 对方 / Tail-W4, > 1 表示 Tail-W4 更快）。按 step 配对（同一批 prompt），多个 seed 合并；区间分别来自逐步独立重抽和长度 4 的滑动块重抽。

对比	比值	95% CI (逐步)	95% CI (块)	seeds	显著
Fixed $K = 3000$	1.049	[0.986, 1.115]	[0.990, 1.113]	42	否
Fixed $K = 5000$	1.003	[0.959, 1.051]	[0.958, 1.040]	42,43	否
Fixed $K = 7000$	1.019	[0.978, 1.061]	[0.973, 1.066]	42	否
Fixed $K = 9000$	1.013	[0.965, 1.068]	[0.962, 1.052]	42	否

Table 8: 各 ablation 相对 Tail-W4（比值 = ablation / Tail-W4, > 1 表示该组件有用）。按 step 配对（同一批 prompt），多个 seed 合并；区间分别来自逐步独立重抽和长度 4 的滑动块重抽。

对比	比值	95% CI (逐步)	95% CI (块)	seeds	显著
No online update	1.047	[1.003, 1.095]	[1.012, 1.084]	42,43	是

Table 9: 归组修复本身（比值 = 旧归组 / 修复后，同 seed）。按 step 配对（同一批 prompt），多个 seed 合并；区间分别来自逐步独立重抽和长度 4 的滑动块重抽。

对比	比值	95% CI (逐步)	95% CI (块)	seeds	显著
Tail-W4	0.974	[0.933, 1.017]	[0.936, 1.010]	42,43	否

修复归组之后，seed 42 从 259.3s 变成 271.9s（旧/新 = 0.972 [0.930, 1.016]），相对 $K=5000$ 只打平（1.001，两个 seed 配对）。原因与 FSDP2 上看到的同一个机制（第 7 节）：2750 组终于学到“切过去之后更长”，决策逃到了由未学习的 uniform-W4 组定价的 2250，seed 42 有 58% 的 rollout、seed 43 有 70% 停在 ≤ 2750 。FSDP2 下地形平，逃过去不吃亏；Megatron 下早切代价大（seed 42 同批 prompt、实测完整 step： $K=1000$ 比 $K=5000$ 慢 8.8%， $K=3000$ 慢 5.4%；始终停在 2750 的 No online update 比 $K=5000$ 慢 6.8%、比旧代码 Tail-W4 慢 11.4%），于是变慢。

旧代码之所以没掉进去，是因为 bug 恰好把 2750 附近切换的数据送进了 cut 0 组，让它也变悲观，堵住了这个出口：两个错误互相抵消。这也说明只修一半是危险的。

8.4 第二个修复 (cc1eea4e)：不在第一个 tail cut 之下切换

第一个 tail cut 之下，校准没有“BF16 前缀之后再切到 W4”的续跑数据；那里的切换由“从第一个 token 起就是 W4”的另一种过程定价，属于外推到错误的分布。cc1eea4e 把候选切换点限制在第一个 tail cut（4B 为 2750）及以上，watcher 默认开启，`--min-switch-frontier 0` 可关闭。用 a703ca55 那条 run 自己的状态离线重放：原来计划 2250 的修订点，加下限后计划 7250–7500，其余修订点不变；这与 Megatron 固定 K 扫描的平坦带（5000–13000）一致。55 个单元测试通过，新增一个测试复现了“不设下限就逃到第一个 cut 之下”。

加下限之后的实测结果见第 9 节：它没有解决问题。

9 新代码 (cc1eea4e, Megatron) 的结果

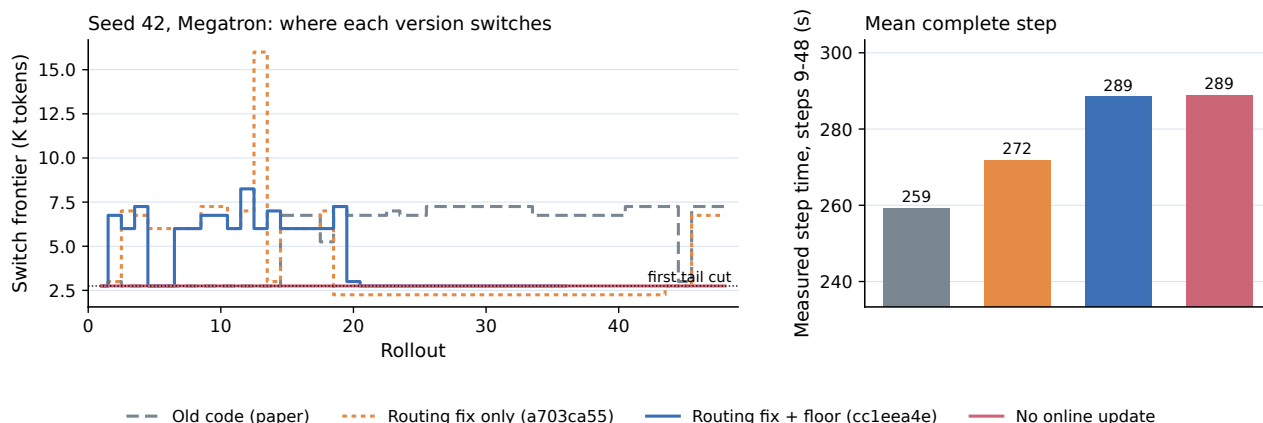


Figure 5: Seed 42, Megatron: 四个版本的切换点与平均 step 时间。旧代码、只修归组、No online update 均为 steps 9-48; 加下限的新代码截至报告时为 steps 9-35。

结论：新代码没有让 4B 变好，seed 42 上显著变差。逐 seed 与同 seed 的旧代码、固定 $K=5000$ 配对（新代码截至报告生成时的步数；seed 43 的新代码 run 在第 31 步被停掉，把 GPU 4-7 让给 Phi 训练，这里用它已完成的部分）：

新代码 Tail-W4 相对	seed 42 (27 步)	seed 43 (23 步)	seed 44 (15 步)
旧代码同 seed (比值 = 旧 / 新)	0.919 [0.851, 0.988]	0.961 [0.879, 1.037]	1.031 [0.903, 1.147]
只修归组同 seed	0.964 [0.902, 1.026]	1.000 [0.924, 1.086]	—
固定 $K=5000$ 同 seed	0.952 [0.888, 1.009]	1.004 [0.909, 1.118]	—

比值 < 1 表示对方更快。停在下限 2750 的 rollout 比例：新代码 seed 42 为 51%、seed 43 为 19%、seed 44 为 4%（旧代码 seed 42 为 27%）。新代码的三个 ablation 只跑到第 30-33 步就被停掉；就这部分看，No downstream cost 反而比完整版快 (0.896 [0.816, 0.973])，说明问题出在学习把决策推向早切，而不是某个组件缺失。

发生了什么 (图 5)。归组修好之后，在线学习终于能改变给 2750 附近切换定价的组，但它学到的结果把决策往早切的方向推：只修归组时逃到第一个 cut 之下的 2250，加了下限之后就贴在下限 2750，和从不更新的 No online update 落在同一个位置、同样的 step 时间。旧代码里这个推动恰好被 bug 挡住：定价组学不到数据，决策停在它校准后更新得最好的位置 (6750-7250)，而那里正好处在实测的平坦带内。根因是在 BF16 线（切换之后看不到 BF16 的尾巴，只能从切换点之前的数据外推）还是在 W4 组的单参数学习上，今晚没有时间查清，需要单独的实验。

建议。

1. **论文保留现有代码 (f8cf7f21) 的结果**，不要换成新代码。今晚已把现有代码的 ablation 补到第二、第三个 seed（队列中，见附录），配合第 8.2 节的表格回应 W6。
2. 在局限性里如实说明：在线更新的归组存在一个实现问题，修复后暴露出学习会推动早切的倾向；现有结果依赖于当前的实现。是否写入由你决定，但评审一旦要代码复现，这一点会被发现。
3. 分支 rollout-precision-scheduler-clean-last-try 的三个提交保留作为后续工作的起点，不合并进论文使用的代码。

10 W2 检验：实测 downstream 是否被污染

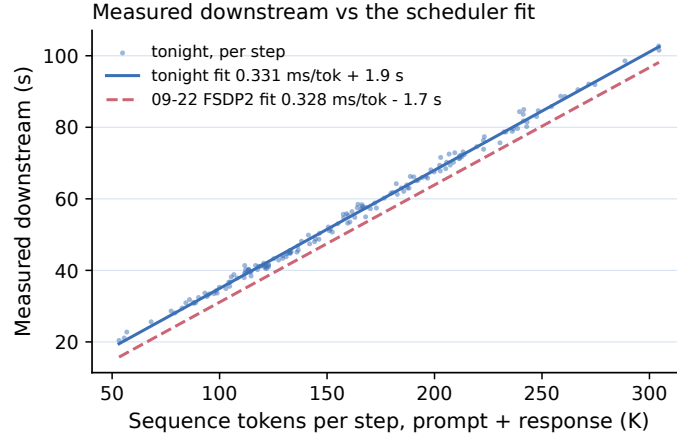


Figure 6: 今晚 FSDP2 run（运行期间不调用 `nvidia-smi`）每一步的实测 downstream（log-prob + 训练 + 权重同步）对序列 token 数。

FSDP2 没有 reference 那一遍，实测 downstream 就是 log-prob + 训练 + 权重同步。拟合结果：

	斜率 (ms/token)	截距 (s)
今晚 FSDP2 run（运行期间不调用 <code>nvidia-smi</code> ，119 步）	0.331	+2.1
9 月 22 日 FSDP2 run（120 步）	0.332	+1.8
调度器使用的回放拟合	0.328	-1.7

三者斜率相差不到 1%；截距之差正好是权重同步（平均 4.5s），扣掉后是 -2.6s。今晚与 9 月 22 日的测量一致，**没有看到 run 内计时被污染的迹象**。Megatron 那一轮里，实测与 Table 1 拟合之差等于 reference 那一遍加权重同步（第 2 节），同样不需要用“测量污染”来解释。

A 今晚的 run 清单

目录都在 `/data/huanchen/ps_runs/`。后缀：fstdp = FSDP2、KL 关；mg = Megatron、KL 开、代码 a703ca55；mgf = Megatron、KL 开、代码 cc1eea4e（加下限）。

目录	verl 提交	状态
eff2_qwen3_5_4b_beta0_s43_mg	a703ca55	COMPLETE
eff2_qwen3_5_4b_const_s42_mgf_STOPPED_for_phi	cc1eea4e	运行中 30/48
eff2_qwen3_5_4b_full_s42_fstdp	a703ca55	COMPLETE
eff2_qwen3_5_4b_full_s42_mg	a703ca55	COMPLETE
eff2_qwen3_5_4b_full_s42_mgf	cc1eea4e	运行中 35/48
eff2_qwen3_5_4b_full_s43_fstdp_ABORTED_requeued	a703ca55	停止（已被取代）
eff2_qwen3_5_4b_full_s43_mg	a703ca55	COMPLETE
eff2_qwen3_5_4b_full_s43_mgf_STOPPED_for_phi	cc1eea4e	运行中 31/48
eff2_qwen3_5_4b_full_s44_mgf	cc1eea4e	运行中 24/48
eff2_qwen3_5_4b_k2750_s42_fstdp	a703ca55	COMPLETE
eff2_qwen3_5_4b_k2750_s42_mg	cc1eea4e	运行中 24/48
eff2_qwen3_5_4b_k5000_s42_fstdp	a703ca55	COMPLETE
eff2_qwen3_5_4b_k5000_s43_mg	a703ca55	COMPLETE
eff2_qwen3_5_4b_k5000_s44_mg	cc1eea4e	运行中 13/48
eff2_qwen3_5_4b_k7000_s42_fstdp	a703ca55	COMPLETE
eff2_qwen3_5_4b_lambda0_s42_mg_ABORTED_superseded	a703ca55	停止（已被取代）
eff2_qwen3_5_4b_lambda0_s42_mgf_STOPPED_for_phi	cc1eea4e	运行中 33/48
eff2_qwen3_5_4b_uncond_s42_mgf_STOPPED_for_phi	cc1eea4e	运行中 31/48

复用的已有 run（行为不受今晚代码改动影响）：Megatron 的 BF16 seed 42/43、固定 K 扫描、阈值 t 扫描、uniform W4、No online update seed 42 (`b32c24_qwen3_5_4b_*`)；FSDP2 的 BF16 与 No online update seed 42

(b32c24_qwen3_5_4b*_fsdp)。脚本:/tmp/claude-1004/-data-huanchen/4d869d23-67f5-4e71-b172-03a3271d9257/scr
(run_lt.sh 启动、analyze.py 数字、figs_*.py 图、make_tables.py 表)。